Minecraft 无规则服务器·多智能体作战体系设计（v2）

设计日期：2026-08-08 | 项目：`minecraft-agents` | 目标场景：无规则（anarchy）服务器 | 本文取代 v1《架构评审》中"仅隔离测试服的安全执行底座"定位，将系统重定位为**多智能体作战集群**，但保留 v1 已验收的全部技术骨架。

一页结论：目标从"安全测试底座"转为"无规则服作战集群"后，现有架构骨架**仍然成立**——Mineflayer Runtime、确定性 L0 反射层、技能白名单、AI 旁路、协议先行都可直接复用。真正新增的是三样东西：**七类作战角色**（侦察/突袭/防御/战斗/建造/农场/后勤）、**三层 AI 调度**（战略/战术/反射），以及一整套**正式服现实约束**（正版账号认证、封号风险、存活即金钱、网络延迟）。
最关键的一条设计红线：**敌我识别是确定性白名单规则（白名单外一律视为敌人），AI 只决定"去哪、要不要发起战斗"，代码决定"打谁、怎么打"。**战斗机制（弹出图腾、水晶摆放/引爆时序、换甲）与敌我识别都必须在 tick 级由代码完成，AI 反应速度（100ms~数秒）根本不可能参与实时战斗，且 AI 判敌我会犹豫、会被话术欺骗。这条原则恰好是项目 v1 已有的"模型不碰 tick"边界，在作战场景下从"良好实践"升级为"生死线"。

一、定位转变：什么保留，什么新增

维度	v1（隔离测试底座）	v2（无规则服作战集群）	结论
连接目标	仅回环/内网隔离服，offline 认证	公网正式服，online-mode 正版账号，多小号	**根本性改变**
L0 安全层	防溺水/岩浆/坠落，边角能力	存活=保住全部装备，是核心资产，"死前断线"是刚需	**升级为核心**
技能框架	1 个 movement.goto	数十个作战/生产技能，分角色	**大幅扩展，框架复用**
AI Router	P2 远期旁路	提前到核心，负责角色分工与战略	**提前但边界不变**
多 Bot 协调	P1.4 远期	集群作战的前提，提前	**提前**
基地失陷撤离	P2.5 远期边角	被突袭是常态，撤离协议是必备	**升级为核心**
协议先行 / fail-closed / 确定性优先	已验收	不变，且更重要	**直接复用**

开发纪律不变：即便终点是正式服，开发和验收仍**先在隔离测试服、再在私有测试 anarchy 服、最后才上目标服。**这是 v1 路线图既有的"先隔离、后仿真、最后正式"原则，作战场景下更不能跳过——一个未验证的战斗 bot 上正式服，等于送装备加送封号。

二、核心设计原则：三层职责划分

> **敌我识别是确定性白名单，AI 只决定去哪和要不要打，代码决定打谁和怎么打。**把一切按"允许的反应延迟"分到三层，延迟越低越必须是纯代码。

层	反应延迟	由谁做	负责什么
战略层	分钟级	大模型（云或大内存本地）	总目标、角色分配、突袭/防御/扩张决策、去哪个区域、是否发起战斗（**不做敌我判断、不做威胁评估**）
战术层	秒级	小模型（本地低延迟）	角色内的临场选择：先打哪个箱子、从哪个方向突入、要不要撤
反射层	tick 级 (<50ms)	纯确定性代码，**零 AI**	**敌我识别（白名单外一律视为敌人）**、弹图腾、水晶摆放/引爆、换甲、自动进食、低血撤退、危险断线、寻路避障

敌我识别规则（硬编码，不经 AI）：维护白名单（本方 Bot、盟友、用户显式标记的中立玩家），**白名单之外的一切玩家一律视为敌人、可交战**；是否此刻真打由确定性存活逻辑决定（打得过就打、打不过先撤），不由 AI 判敌我。白名单只能由用户显式配置，AI 无权增删。这样既躲开 AI 判敌我的犹豫与被话术欺骗，也让"非友即敌"契合无规则服的现实。

这层划分直接映射到现有代码：反射层就是已验收的 `packages/safety`（L0）+ `packages/skills` 执行器；战术/战略层是 P2 的 AI Router，输出只能是"经 Schema 校验的白名单技能调用 JSON"，永远不下发原始动作。**任何让 AI 直接控制战斗动作或敌我识别的设计都要在评审中否决**——不是出于安全洁癖，而是它必然打不过纯代码的对手。

三、七类作战角色

每个角色标注：绿色=AI 决策部分，橙色=确定性代码部分，以及所需技能。角色是"能力集合"，一个 Bot 可按战略层分配临时担任某角色。

1. 侦察 / 探索 Scout

职责：沿主干道（地狱顶层公路、坐标轴高速）远程旅行，记录基地/物资仓/玩家坐标，覆盖地图空白区，跟踪玩家找到其基地，鞘翅+烟花飞行，地狱门 8 倍距离穿梭。

AI 决策：探索方向与覆盖策略、某结构是否值得上报、是否跟踪某玩家。
确定性：公路循迹、避障、鞘翅飞行力学、自动进食、危险断线、坐标记录。

技能：`travel.highway` · `fly.elytra` · `scan.chunks` · `track.player` · `log.coordinate`

产出：写入共享世界记忆（基地/仓库坐标、玩家活动规律）——正是 P2.1 pgvector 长期记忆的输入源。侦察是整个集群的"眼睛"，突袭和防御都依赖它的数据。

2. 突袭 / 破坏 Raider

职责：攻破基地——黑曜石开采（附魔镐/水晶/TNT）、末影水晶爆破、凋灵拆墙、岩浆/水破坏、翻箱取物、拆除方块。

AI 决策：突入方式（挖进去 vs 炸开）、何时放弃撤退。
确定性：敌我识别、目标优先级（先抢哪个箱子）、水晶摆放机制、TNT 引爆时序、挖掘模式、背包管理、撤退触发。

技能：`mine.obsidian` · `place.crystal` · `ignite.tnt` · `spawn.wither` · `loot.chest` · `breach.wall`

依赖：侦察提供坐标、农场提供 TNT/水晶/黑曜石、战斗角色护航。突袭是集群资源消耗最大的角色，必须有稳定的战争经济支撑。

3. 防御 Defender

职责：守卫己方基地，探测入侵者（范围内玩家实体），响应威胁，维护围墙/边界，向集群报警，记录威胁。

AI 决策：迎战 vs 躲藏 vs 撤离、是否触发"基地失陷协议"。
确定性：敌我识别（白名单外即入侵者）、入侵探测、报警派发、边界巡逻路径、撤退。

技能：`patrol.perimeter` · `detect.intruder` · `alert.swarm` · `seal.entrance`

关键联动：触发 `COMPROMISED` 状态后转入撤离协议（见第六节）——核心物资撤离、在≥10000 格外重建。这是 v1 路线图 P2.5 的设计，在作战场景中是必备而非边角。

4. 战斗 / PvP Combatant

职责：水晶 PvP、图腾管理、换甲、武器选择、锁定目标、拉近/脱离。护航突袭队，抵御攻击者。

AI 决策：**仅**是否交战/脱离、撤退阈值——战略级。
确定性（必须）：敌我识别与目标选择（白名单外即目标）、图腾自动弹出（亚 tick 反应）、水晶摆放/引爆时序、护甲耐久换装、快捷栏管理、瞄准。

本项目最重要的技术判断：战斗机制必须 100% 确定性代码，且要跑在延迟最低的路径上。AI 的最快反应也在 100ms 以上，而水晶 PvP 的图腾弹出、连锁爆破需要 <50ms 反应——让 AI 参与战斗动作等于必输。这也是下文 Rust/网络延迟讨论的核心动因。

打法来源：参考开源、自研反射层。水晶 PvP、AutoTotem、AutoArmor、Surround 等算法以成熟开源实现（Meteor Client、Baritone、mineflayer-pvp）为**打法参考**，在本项目自研，三条硬线：① **只学机制与参数、重写为本项目 TS，禁止逐行翻译/复制 GPL 源码**（算法思路不受版权保护，逐行翻译属衍生作品受 GPL 约束）；② Meteor 等为 GPL-3.0，**仅在不分发时无约束，分发前必做许可评审**，默认按本地自用处理；③ **继续用 Mineflayer 无头 bot，不引入 MC 客户端模组栈**（Meteor 是 Java 客户端 Fabric 模组，架构不兼容且资源开销涨一到两个数量级）。Mineflayer 无头能否达到 <50ms 时序是**未验证假设**，由 M3 单独实测硬化；扛不住才评估双栈或 Rust。

技能：`combat.crystal` · `combat.melee` · `manage.totem` · `manage.armor`

5. 建造 Builder

职责：建基地、围墙、隐蔽基地（地下/伪装）、自动化设施骨架（农场框架、红石）、黑曜石地堡。

AI 决策：基地选址（隐蔽、远离公路）、布局、建什么。
确定性：方块摆放模式、蓝图执行、脚手架。

技能：`build.blueprint` · `place.blocks` · `build.wall` · `build.hidden-base`

6. 农场 / 资源 Farmer（战争经济引擎）

职责：自动化农场——火药（爬行者/女巫塔，供 TNT）、黑曜石生成、作物/食物、经验、材料（圆石/铁）。为突袭供 TNT/水晶原料，为全员供金苹果/食物。

AI 决策：按战争需求决定优先生产什么（要突袭就多囤火药和水晶原料）。
确定性：农场运转循环、收割、补种、收集。

技能：`farm.crop` · `farm.mob` · `generate.obsidian` · `smelt` · `craft.item`

没有稳定的农场经济，突袭和战斗就是无源之水。这个角色决定集群能持续作战多久。

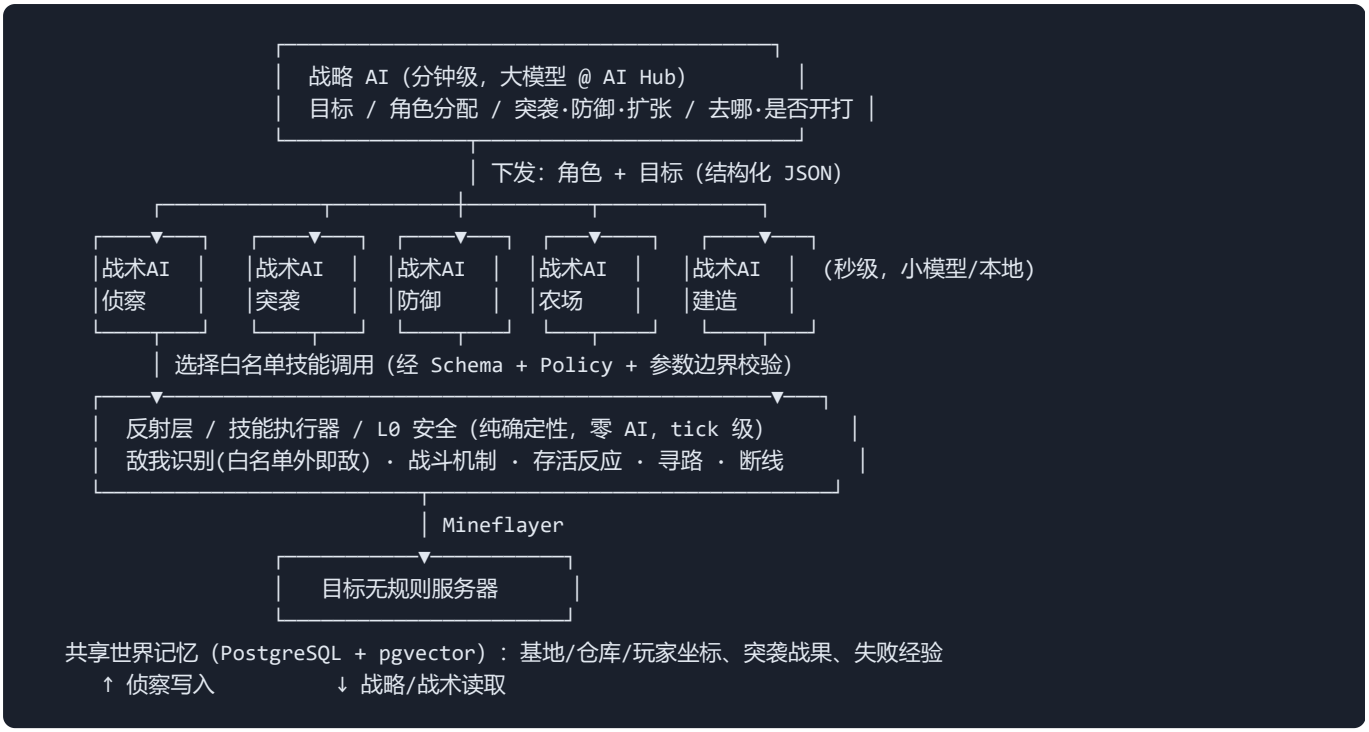
7. 后勤 / 仓储 Logistics

职责：物资仓管理、潜影盒打包、物品分类、在农场/基地/前线间补给运输、给 bot 补货。

确定性为主：`store.stash` · `pack.shulker` · `resupply` · `sort.items`

作战集群里"死一个 bot 掉一套装备"是常态，后勤负责让损失可快速补充，把前线和仓储解耦。

四、AI 调度体系



调度要点

- **角色分配是动态的**：战略 AI 根据战况把 Bot 在角色间调度——被突袭时把突袭队召回转防御，资源短缺时增派农场。
- **AI 宕机不致命**：AI Router 是旁路。战略 AI 掉线，Bot 退回"维持当前角色 + 反射层存活"，不会失控——这是 fail-closed 在作战场景的体现。
- **模型输出永不直接执行**：战略/战术层输出的都是"技能 ID + 参数"，经协议校验和 Policy 白名单后才由执行器调用。模型说不出反射层没实现的动作。
- **共享世界记忆是集群智能的关键**：单个 Bot 的视野有限，价值在于把侦察、突袭、失败经验汇聚成集群共享的知识库（对应 P2.1）。

五、Rust vs TypeScript 重新评估

作战场景让 v1 中"暂不需要 Rust"的结论出现了两个**更可能触发**的位置，但触发前提仍是"先测量、且 Mineflayer 锁定 Node"这个硬约束。

模块	v1 判断	v2 判断（作战场景）	触发条件
寻路 / 世界几何	远期可选	**明显增强**：探索是 baritone 级远距离寻路，纯 CPU 密集	多 Bot 探索时寻路阻塞事件循环（延迟 >50ms 可测）→ 独立 Rust 寻路服务，JSON/gRPC 接口
战斗反射时序	未提及	**新增关注**：Node 事件循环抖动可能让你输掉战斗/丢装备	先测量战斗路径的事件循环抖动；**但 Rust 战斗必须离开 Mineflayer（换 azalea），当前不成熟**，所以短期仍用进程内紧凑确定性 Node，仅在证明不足且 azalea 成熟后再议
其余全部	TS	TS	协议、Coordinator、任务、Dashboard、农场/建造逻辑都是 I/O 或轻计算，无 Rust 收益

不变的铁律：不在有性能测量数据前写任何 Rust。作战目标只是让"寻路服务"这一个位置从"远期可选"变成"较可能需要"，不改变"先测量、Rust 只放叶子位置、绝不进控制平面"的原则。

六、正式服现实约束（新增核心章节）

这是从测试底座转向正式服作战时**绕不开、且 v1 完全没有**的部分。诚实对待，不假装已解决。

6.1 账号与认证

- 正式服是 online-mode，需**正版微软账号**（OAuth token），取代 v1 的 offline-only。集群=多个小号（正版约 $30/个）。
- **凭据纪律不变**：token 加密存储，绝不进仓库、日志、聊天；这是 v1 密钥纪律的延续。每个账号一套独立凭据。
- Runtime 需新增 online-mode 认证适配，且这条路径必须与"隔离测试 offline"路径并存、由 ServerProfile 显式区分，防止误把测试逻辑带上正式服。

6.2 封号风险（务必如实告知使用者）

- 多数服务器反作弊会封 bot，账号即金钱损失，集群化放大风险。2b2t 类明确允许 bot 的服是例外，但仍会因生存区 fly/reach 等作弊封号。
- 缓解手段：**类人化移动与节奏**（避免瞬移/完美瞄准/不可能操作）、每账号限速、行为随机抖动、接受残余风险。
- **红线**：不把"反封号"当成已解决问题写进文档；缓解=降低明显 bot 特征 + 遵守目标服规则，不等于免疫。是否接受封号风险是使用者的决定，系统只做到"减少明显破绽 + 死号可快速替换"。

6.3 存活即金钱 & 基地失陷协议

- 正式服死亡=掉光装备（巨大成本），L0"死前断线/存活优先"从边角升级为最高优先级反射。
- 贵重装备平时存仓、作战按需领取；bot 感知危急时优先断线保号而非硬拼。
- 基地被突袭是常态 → `COMPROMISED` 状态、证据记录、核心物资撤离、在≥10000 格外新址重建（P2.5 设计直接启用）。

6.4 网络延迟（作战特有，易被忽视）

> 战斗 bot 对**到服务器的网络延迟**极其敏感——高 ping 会直接输掉水晶 PvP。因此**战斗/突袭 bot 不应跑在高延迟的家用网络上**，而要放在靠近目标服务器的 VPS/云主机。农场、建造、后勤这类非实时角色对延迟不敏感，可跑在家里的基础设施节点。这决定了集群是分布式部署，不是一台机器全包。

七、硬件角色映射

机器	角色	说明
迷你主机 / NUC（家用）	基础设施节点	测试服 + PostgreSQL/Redis + Coordinator/服务进程 + 农场/建造/后勤 bot（延迟不敏感）
大内存 NUC（356H+96G）/ DGX Spark	AI Hub	战略+战术模型推理；低频调用，正好匹配核显/统一内存的吞吐特性；DGX Spark 是 70B+ 私有化推理的正解
靠近目标服的 VPS/云主机	前线节点	战斗/突袭/侦察 bot，吃低 ping；一账号一进程，轻量但要低延迟

这正好落实 v1 硬件讨论的结论：NUC 管"今天需要的基础设施"，DGX Spark 管"v3 验证成功后的私有 AI"，而作战场景新增了"前线 VPS"这一维度。

八、MVP 重定义（面向作战，仍要小而可证）

MVP：单个 Bot 在测试服存活、旅行、独立完成一个非战斗角色任务（先做侦察或农场），装备可暂存、危险可断线、全程可观测。战斗作为独立的、单独硬化的里程碑，绝不与 MVP 混在一起。

分阶段路线

阶段	内容	验收
M0 闭环底座	完成 v1 遗留的 P1.3.2 持久任务闭环 + Coordinator 任务调度接入 + 审计落库（这些无论作战与否都要）	杀进程任务不丢；Dashboard 建任务能驱动 Bot 动作
M1 存活与旅行	online-mode 认证适配（与测试 offline 并存）；L0 存活升级（死前断线、装备暂存）；`travel.highway` + 鞘翅飞行	Bot 在私有测试 anarchy 服长时间存活、能远程旅行、危险自动断线保号
M2 首个生产角色	侦察角色全链路（探索+坐标记录+写入共享世界记忆）**或**农场角色（一个自动农场稳定产出）——二选一先做	该角色在测试服连续运行数小时无人干预，产出真实可查
M3 战斗反射（独立硬化）	确定性水晶 PvP / 图腾 / 换甲，在隔离 PvP 测试环境反复验证反应时序	在受控 PvP 测试中稳定弹图腾、连锁爆破；事件循环抖动实测达标
M4 多角色协调	Coordinator 进程编排（拉起/回收多 Bot）+ 角色分配 + 共享世界记忆 + 分布式部署（家用+VPS）	侦察→突袭→撤退的多 Bot 协同在测试服跑通
M5 收敛重构	v1 评审的 secure-link 抽取 + 千行文件拆分（趁规模可控）	重构前后测试行为一致
M6 智能层 & 上正式服	AI Router 接战略/战术（先云 API 验证价值，再决定本地化）；灰度上目标服	先小规模灰度、接受并评估封号风险后再扩张

顺序要点：① 底座（M0）和存活（M1）是一切的前提，不能跳；② 战斗（M3）单独硬化、绝不与生产角色混做；③ 上正式服（M6）永远在最后，且先灰度；④ AI 智能层同样先用云 API 验证"分工调度到底提升多少"，再决定是否上本地大模型——不因手里有 DGX 就提前。

九、给 Codex 的要点

1. 目标是无规则服多智能体作战集群，但**开发/验收严格遵循"隔离测试服 → 私有测试 anarchy → 灰度正式服"**三段式，不跳级。
2. 牢记三层职责：**敌我识别是确定性白名单（白名单外一律视为敌人），AI 只决定去哪/是否开打，代码决定打谁/怎么打。**任何让模型直接控制战斗、tick 动作或敌我识别的设计一律否决。
3. 战斗（M3）必须作为独立里程碑单独硬化，先在隔离 PvP 环境验证反应时序，达标前不上任何真实对抗。
4. 战斗打法参考开源实现（Meteor/Baritone/mineflayer-pvp）但**只学机制参数、重写为本项目 TS，禁止逐行翻译/复制 GPL 源码**；GPL 仅不分发时无约束，分发前必做许可评审；不引入 MC 客户端模组栈，继续用 Mineflayer 无头 bot。

5. online-mode 认证与隔离测试 offline 认证**并存并由 ServerProfile 显式区分**，严禁把测试捷径带上正式服；账号凭据加密、不落仓库/日志。
6. 封号风险如实记录，不假装已解决；"类人化"是降低破绽不是免疫。
7. 其余 v1 铁律不变：协议先行、fail-closed、不重写已验收模块、不无测量写 Rust、`pnpm check` 全绿才算完成、不冒充验收。

本文基于 2026-08-08 代码库状态与用户明确的"无规则服作战集群"目标编写，取代 v1《架构评审》的项目定位部分；v1 中关于协议、fail-closed、验收文化、Rust 触发原则的技术结论继续有效。所述作战角色为 Minecraft 游戏内玩法（anarchy 服允许的探索、突袭、PvP、建造、自动化），系统仅在游戏范围内运行。